

Credit Card Fraud Detection with Vertex AI AutoML

This is my shorter portfolio version of a Spring 2025 EAI6020 project. I used Google Cloud Vertex AI AutoML to study an imbalanced credit card fraud dataset and turn the result into a business decision problem. My main work was dataset selection, business cost framing, AutoML setup, threshold review, and result interpretation.

Vertex AI AutoML

Google Cloud

PR Curve

Threshold Tuning

Fraud Detection

SIMPLE NOTE

This PDF rewrites my original assignment into a cleaner case-study format. The original model training was done in the Vertex AI interface, so this project is stronger as a cloud AI portfolio piece than as a coding-heavy ML repo.

BUSINESS PROBLEM

Fraud teams need to catch risky transactions without creating too many false alarms for real customers. Because fraud cases are rare, a default accuracy view is not enough, and threshold choice becomes a business decision.

PROJECT INFO

At a glance

Course: EAI 6020 - AI Systems Technology

Term: Spring 2025

Type: Individual assignment

Team: N/A

Original deliverables: report PDF, module slide, sampled CSV

Dataset: Credit Card Fraud Detection

Rows / columns: 20,000 rows, 7 columns

Main tools: Vertex AI AutoML, Kaggle, Google Cloud

Model note: AUC PR objective; selected threshold 0.75; PR AUC 0.989

Project Snapshot

The original assignment asked me to train an AutoML model, evaluate it with the precision-recall curve, and set a useful score threshold. This portfolio version focuses on the parts that best show my thinking: business framing, data profile, cloud constraint handling, evaluation screenshots, and the final threshold decision.

BUSINESS FOCUS

Fraud screening trade-off

Missing fraud costs money, but false alarms also hurt customer experience.

DATA SCALE

20K working sample

I moved from 100K to 20K after a quota issue and kept the 1% fraud ratio.

METHODS USED

AutoML + PR review

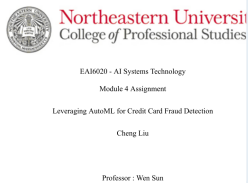
AutoML training, precision-recall review, threshold slider, and feature attribution.

ORIGINAL DELIVERABLES

Report + screenshots

The original work was documented through the report PDF and Vertex AI screens.

Original source materials consolidated here



Original assignment report

EAI6020: AI Systems Technology

Course module slide

I reused the original report screenshots and rebuilt the explanation in a shorter portfolio story. This also fits the course theme of AI adoption, evaluation, and business alignment.

My role in the project

- Selected the fraud dataset and framed the business use case.
- Defined false negative and false positive cost assumptions.
- Ran Vertex AI AutoML and adjusted the plan after a quota failure.
- Reviewed PR results, threshold trade-offs, and feature importance.

Contribution note

This was an individual assignment. The model training itself was done in the Vertex AI UI, and this portfolio PDF keeps that boundary clear.

Problems, Data, and Analysis Plan

I treated this as a small fraud-operations decision case. The goal was not only to get a high score. The bigger question was how to balance missed fraud losses against customer friction from false alarms in a rare-event setting.

PROJECT PURPOSE

I wanted to test whether a cloud AutoML workflow could support a fraud classification task and whether the model output could be translated into a simple operating decision. I also wanted to see how evaluation changed when the dataset was highly imbalanced.

Dataset summary

Details

Main table	Public credit card transaction data with fraud label
Second table	N/A
Merge key	N/A
Working data	20,000-row CSV used for Vertex AI tabular classification
Missing values	None in the uploaded sample used for this portfolio summary

STAKEHOLDERS

Fraud and risk operations, customer support, customer experience, and an AI/product owner who would decide how aggressive the screening system should be.

MAIN QUESTIONS

Can AutoML separate fraud from normal cases?
Why is PR better than accuracy here?
What threshold works better than 0.5?
What do the screenshots and feature attribution tell me?

MY WORKFLOW

choose dataset -> attempt full training
-> quota failure -> make stratified 20K sample -> run AutoML with AUC PR -> review threshold slider -> summarize trade-offs

SELECTED CODE - SAMPLE DATA AUDIT

```
import pandas as pd

df = pd.read_csv(
    "credit_card_fraud_sample_20k.csv.csv",
    parse_dates=["TransactionDate"]
)

fraud_rate = df["IsFraud"].mean()
type_mix = df.groupby("TransactionType")["IsFraud"].mean()
amount_stats = df.groupby("IsFraud")["Amount"].agg(
    ["count", "mean", "median"]
)
```

This is not the original training code. I used it in this portfolio edition to profile the uploaded sample data and verify the rare-class structure.

LAB / CODE NOTE

The original model training and threshold review were done inside the Vertex AI AutoML interface. No custom model notebook was part of the original submission, so I use a small Python audit here only to support the portfolio summary.

EXAMPLE FIELDS I USED

TransactionID, TransactionDate, Amount, MerchantID

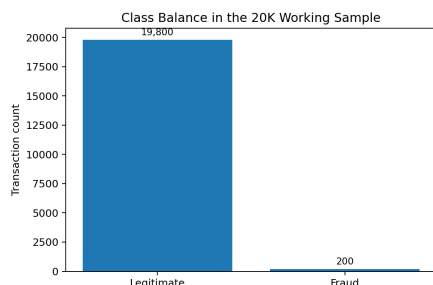
TransactionType, Location, IsFraud

Target column: IsFraud

Key Findings and Visual Evidence

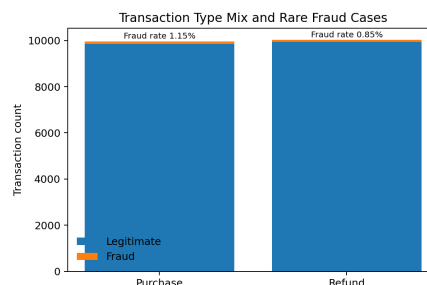
The working sample was clean and easy to profile, but it also showed why this task is hard: fraud cases were only 1% of all rows. That made threshold choice and evaluation logic much more important than plain accuracy.

Class balance in the working sample



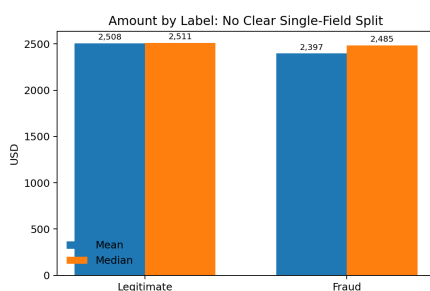
The uploaded sample contained 19,800 legitimate transactions and only 200 fraud cases.

Transaction type mix and rare fraud cases



Purchase and refund counts were almost even, but purchase showed a slightly higher fraud rate.

Amount by label



Average and median amounts were not far apart across labels, so amount alone was not enough to separate fraud from normal cases.

WHAT THESE CHARTS TOLD ME

- The class imbalance was the main evaluation challenge.
- The uploaded sample had no missing values and no duplicate transaction IDs.
- Fraud did not stand out from one simple field like amount alone.
- This made PR-focused evaluation and threshold review more appropriate.

Quick business read

The model needed to optimize rare-case detection, not just majority-class correctness.

Methods, Threshold Logic, and Vertex AI Evidence

Vertex AI gave strong ranking performance, but the default threshold was not a good operating point. This was the most useful lesson from the assignment: model evaluation still needed human judgment and business context.

SELECTED CODE SNIPPET

```
false_negative_cost = 500
false_positive_cost = 50

def cost_view(missed_fraud, false_alerts):
    return (
        missed_fraud * false_negative_cost
        + false_alerts * false_positive_cost
    )

# My portfolio takeaway:
# threshold review should support a
# business cost decision, not only a
# default UI setting.
```

I used this logic to explain why threshold choice mattered. In the report, the actual threshold review was done in the Vertex AI interface.

INTERPRETATION

- My first 100K training attempt failed because of project quota limits.
- I rebuilt the run with a stratified 20K sample and kept the original 1% fraud ratio.
- The successful run used **IsFraud** as the target and **AUC PR** as the optimization objective.
- The model looked strong overall, but the default 0.5 threshold still under-served the rare fraud class.

Vertex AI evaluation and feature attribution



From left to right: evaluation details at the default threshold, precision-recall / ROC / confusion matrix view, and feature attribution where Amount and TransactionType were the strongest visible signals.

Conclusion, Recommendations, and Interview Use

This project was useful because it connected AI tools, business reasoning, and deployment reality. It showed that a strong model score is only part of the story. Threshold choice, cloud limits, and explainability also matter.

MAIN FINDINGS

Vertex AI AutoML produced a strong fraud ranking model on the sampled data.

I moved away from the default threshold and selected 0.75 as a better balance point, with about 96% precision and 82% recall.

RESULT MEANING

For a fraud team, this means the model can screen many risky transactions while keeping false alarms lower than a very aggressive threshold.

RECOMMENDATIONS

Add a threshold-vs-cost table, re-test without TransactionID as a feature, and validate on a less synthetic dataset before claiming deployment readiness.

WHAT THIS PROJECT SHOWS ABOUT MY WORK

- I can translate model evaluation into business decisions.
- I understand imbalanced classification and PR-curve logic.
- I can work with cloud AI tools and adjust when infrastructure limits appear.
- I can present technical results in a concise and readable way.

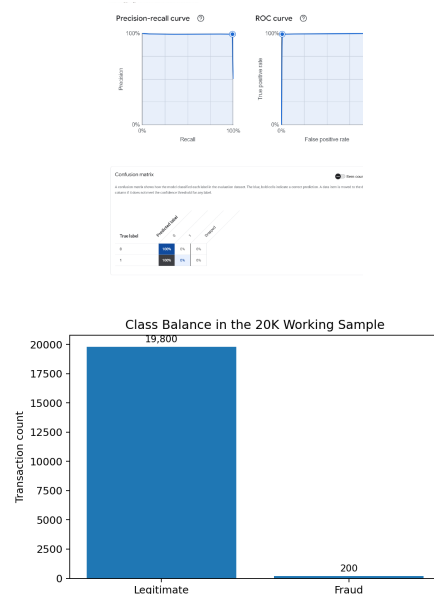
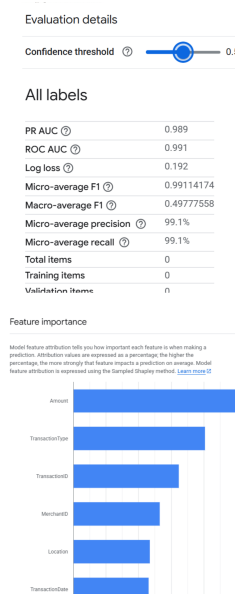
HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

Simple version: In this project, I used Vertex AI AutoML for a credit card fraud case. The key point was not only to get a high score, but to choose a threshold that better balanced missed fraud and false alarms. I also had to handle a quota issue, so I rebuilt the run with a stratified 20K sample and kept the 1% fraud ratio.

Final note

This was an individual course assignment. The most portfolio-ready part is my business framing, threshold reasoning, and clear explanation of what the model could and could not prove.

Quick Visual Gallery



I kept a small gallery because this project was driven by Vertex AI interface outputs, so screenshots were part of the real workflow.